

RATL Specification

RATL (R-based Array Manipulation Language) is a stack-based, esoteric programming language inspired by [MATL](#). It leverages R's powerful statistical and matrix capabilities through a concise, postfix syntax, making it highly suitable for code golf.

Total symbols: 427 (all 2-byte or shorter)

1. Introduction

RATL is implemented in R and uses R's internal data structures (vectors, lists, matrices) to handle stack operations. It translates RATL code into R code, which is then evaluated.

1.1 The Name

The name stands for **R-based Array Manipulation Language**, reflecting its foundation in R and its primary focus on array-based operations.

1.2 Notation

- **\$n**: Represents the n -th element popped from the stack (starting from the top).
 - **stack**: The global list used for storage.
 - **H, G, L, M**: Clipboards for temporary storage outside the stack.
-

2. The Stack and Data Types

RATL is a stack-based language where every operation interacts with a global stack (an R `list`).

Data Types

RATL uses native R types: - **Numeric**: Integers and floating-point numbers (R `numeric`). - **Logical**: Boolean values (`TRUE`, `FALSE`). - **Character**: Strings of text (R `character`). - **List/Cell Array**: Collections of items, which can be of mixed types (R `list`). - **Matrix**: 2D arrays (R `matrix`).

3. Statements and Separators

- **Space:** Acts as a token separator.
 - **Newline:** Ignored by the parser but acts as a token separator.
 - **Comments:** Lines starting with **#** are treated as comments and ignored.
-

4. Literals

Literals are pushed onto the stack as soon as they are encountered.

4.1 Numbers

Standard numeric notation: 1, 10, 3.14, -5.

4.2 Numerical Arrays (Vectors)

Enclosed in square brackets [...]. Elements are separated by spaces. Example: [1 2 3] pushes a numeric vector of length 3.

4.3 Character Arrays (Strings)

Enclosed in single quotes '...'. Example: 'hello' pushes a character vector of length 1.

4.4 Cell Arrays (Lists)

Enclosed in curly braces {...}. Example: {1 'a' [1 2]} pushes an R list containing three different types.

5. Control Flow

RATL uses special symbols for conditional branching and loops.

5.1 Conditional Branching (? ...])

- **Symbols:** ? starts the block,] ends it.
- **If-Else:** ? **truthy** ; **falsy**]. The ; separator distinguishes the “then” and “else” branches.
- **Logic:** Pops the top element. If it's true, it executes the code until] (or ;). If false and ; is present, it executes the code between ; and].

5.2 While/Repeat Loop (" ... ")

- **Symbol:** "
- **Logic:** Starts a loop. At the end of the loop (marked by another "), it pops the top element. If it's true, it repeats.

5.3 For-Each Loop ((...))

- **Symbols:** (and)
- **Logic:** (pops an array/list. It then iterates through each element, pushing it onto the stack and executing the code between (and).

5.4 Infinite Loop (` ... `)

- **Symbols:** ` starts and ends the loop.
- **Logic:** Executes the code within the backticks repeatedly. Use the X (break) symbol to exit.

5.5 Higher-Order Functions (Block Consumers)

These functions pop a {block} from the stack and apply it to data.

Symbol	Name	Description
q	Map	{block} array q — applies block to each element
e	Filter	{block} array e — keeps elements where block returns truthy
y	Reduce	{block} array y — fold left with block
z	Repeat	N {block} z — executes block N times
@	Execute	{block} @ — pops and executes a block immediately

6. Implicit Actions

6.1 Initial Actions

- The stack is initialized as an empty list().
- A connection to `stdin` is opened for input operations.

6.2 Final Actions

- **Implicit Print:** If the stack contains exactly one element, it is printed automatically. If it contains more than one, the entire stack is printed as a list.
 - **Invisible Output:** The final evaluation result is returned invisibly in R.
-

7. Meta-Programming

RATL supports meta-programming — code that generates or manipulates other code at runtime.

7.1 Dynamic Evaluation (ev)

Pops a string from the stack and executes it as RATL code:

```
'3 5 +' ev      → 8
'[1 2 3] s' ev   → 6
'5 fp' ev        → 120
```

7.2 Dispatch List (zD)

Pushes a list of all available symbol names onto the stack:

```
zD      → [+ - * / D w x ...]
```

7.3 Dynamic Dispatch (zS)

Calls a symbol by name (string):

```
3 5 '+' zS      → 8
5 'sq' zS       → 2.236...
```

7.4 Shell Exec (zX)

Runs a shell command and pushes the output:

```
'echo hello' zX  → "hello"
'ls' zX          → file listing
```

7.5 Sort By (zB)

Sorts an array using a block as the key function:

```
[3 1 4 1 5] {0-} zB    → 1 1 3 4 5 (reverse sort)
[1 2 3] {2 *} zB       → 1 2 3
```

7.6 Group By (zG)

Groups array elements by the block's return value:

```
[1 2 3 4 5 6] {2 %} zG    → $0: [2 4 6] $1: [1 3 5]
```

7.7 Scan (zC)

Like reduce, but returns all intermediate results:

```
[1 2 3 4 5] {+} zC      → 1 3 6 10 15
[2 3 4] {*} zC           → 2 6 24
```

7.8 Take While (zT)

Takes elements while the block returns truthy:

```
[1 2 3 4 5] {4 <} zT     → 1 2 3
```

7.9 Drop While (zW)

Drops elements while the block returns truthy:

```
[1 2 3 4 5] {3 <} zW     → 3 4 5
```

7.10 Zip With (zZ)

Applies a block pairwise to two arrays:

```
[1 2 3] [4 5 6] {+} zZ   → 5 7 9
```

8. Symbol Reference

8.1 Arithmetic & Comparison

Symbol	R Code	Description	Stack Effect
%	\$2 %% \$1	Modulo	2 → 1
*	\$2 * \$1	Multiply	2 → 1
+	\$2 + \$1	Add	2 → 1
-	\$2 - \$1	Subtract	2 → 1
/	\$2 / \$1	Divide	2 → 1
<	\$2 < \$1	Less	2 → 1
<=	\$2 <= \$1	LessEqual	2 → 1
=	\$2 == \$1	Equal	2 → 1
>	\$2 > \$1	Greater	2 → 1
>=	\$2 >= \$1	GreatEqual	2 → 1
^	\$2 ^ \$1	Power	2 → 1
lX	xor(\$2, \$1)	Xor	2 → 1
mI	\$2 %/% \$1	IntDiv	2 → 1
\	\$2 \ \$1	Or	2 → 1
~	!\$1	Not	1 → 1

8.2 Stack & Control

Symbol	R Code	Description	Stack Effect
D	list(\$1, \$1)	Duplicate	1 → 2
Ls	length(stack)	Stack Length	0 → 1
U	for(x in \$1) stack[[length(st...	Unpack	1 → 0
i	scan(ratl_stdin, what=charact...	Input	0 → 1
p	ratl_print(\$1)	Print	1 → 0
sC	cat(\$1)	Cat	1 → 0
sM	message(\$1)	Message	1 → 0
sS	stop(\$1)	Stop	1 → 0
sW	warning(\$1)	Warning	1 → 0
w	list(\$1, \$2)	Swap	2 → 2
x	NULL	Delete	1 → 0

8.3 Constants

Symbol	R Code	Description	Stack Effect
In	Inf	Inf	0 → 1
Na	NA	NA	0 → 1
Pi	pi	Pi	0 → 1
s4	logical(0)	EmptyLog	0 → 1
sQ	character(0)	EmptyChar	0 → 1
sZ	numeric(0)	EmptyNum	0 → 1

8.4 Higher-Order Functions

Symbol	R Code	Description	Stack Effect
Fa	apply(\$3, \$2, \$1)	Apply	3 → 1
Ff	Filter(\$2, \$1)	Filter (Func)	2 → 1
Fl	lapply(\$2, \$1)	Lapply	2 → 1
Fm	mapply(\$1, ...)	Mapply	1 → 1
Fn	Find(\$2, \$1)	Find (Func)	2 → 1
Fp	Position(\$2, \$1)	Position	2 → 1
Fq	NULL	Map (evaluator)	0 → 1
Fr	Reduce(\$2, \$1)	Reduce	2 → 1
Fs	sapply(\$2, \$1)	Sapply	2 → 1
Ft	NULL	Filter (evaluator)	0 → 1
Fv	vapply(\$3, \$2, \$1)	Vapply	3 → 1
Fx	NULL	Repeat (evaluator)	0 → 1
e	NULL	Filter (evaluator)	0 → 1
fn	Negate(\$1)	Negate	1 → 1
q	NULL	Map (evaluator)	0 → 1
y	NULL	Reduce (evaluator)	0 → 1
z	NULL	Repeat (evaluator)	0 → 1

8.5 Array Operations

Symbol	R Code	Description	Stack Effect
#	\$2[\$1]	Filter	2 → 1
,	c(\$2, \$1)	Concat Vectors	2 → 1
.	1:\$1	Range 1 to N	1 → 1
..	list(\$2, \$1)	Pair	2 → 1
:	seq(\$2, \$1)	Sequence	2 → 1
O	sort(\$1)	Sort	1 → 1
R	rev(\$1)	Reverse	1 → 1
SH	head(\$1)	Head	1 → 1
SR	rank(\$1)	Rank	1 → 1
ST	tail(\$1)	Tail	1 → 1
XC	cumsum(\$1)	CumSum	1 → 1
Xk	{r <- rle(\$1); list(r\$values, ...}	RLE	1 → 2
c1	cumprod(\$1)	CumProd	1 → 1
cn	is.element(\$2, \$1)	IsIn	2 → 1
dO	order(\$1)	Order	1 → 1
dS	split(\$2, \$1)	Split	2 → 1
dU	unsplit(\$2, \$1)	Unsplit	2 → 1
e1	\$2[[\$1]]	Extract Element [[	2 → 1
en	\$2[[as.character(\$1)]]	Extract Name \$	2 → 1
es	\$2[\$1]	Extract Subset [	2 → 1
fE	head(\$1, 1)	FirstElem	1 → 1
fu	unlist(\$1)	Flatten	1 → 1
hd	head(\$2, \$1)	HeadN	2 → 1
ix	match(\$2, \$1)	IndexOf	2 → 1

Symbol	R Code	Description	Stack Effect
l	length(\$1)	Length	1 → 1
la	tail(\$1, 1)	LastElem	1 → 1
ns	\$2[length(\$2)+1-\$1]	NegSlice	2 → 1
r1	1:\$1	Range1toN	1 → 1
r2	rep(\$2, \$1)	Rep	2 → 1
tb	tabulate(\$1)	Tabulate	1 → 1
t1	tail(\$2, \$1)	TailN	2 → 1
u	unique(\$1)	Unique	1 → 1
un	length(unique(\$1))	UniqueN	1 → 1
vB	pmin(\$2, \$1)	PMin	2 → 1
vD	drop(\$1)	Drop	1 → 1
vF	diff(\$1)	Diff	1 → 1
vG	pmax(\$2, \$1)	PMax	2 → 1
vH	which(\$1, arr.ind=TRUE)	WhichArr	1 → 1
vI	cummin(\$1)	CumMin	1 → 1
vT	matrix(rep(\$3, \$2*\$1), nrow=n...	RepMat	3 → 1
vW	seq(\$3, \$2, length.out=\$1)	SeqLen	3 → 1
vX	cummax(\$1)	CumMax	1 → 1
wh	which(\$1)	Which	1 → 1
zp	unlist(Map(list, \$2, \$1))	Zip	2 → 1

8.6 Matrix

Symbol	R Code	Description	Stack Effect
!	t(\$1)	Transpose	1 → 1
&	\$2 %o% \$1	Outer Product	2 → 1
BT	{idx<-c(1,4,3,2,5); \$1[idx,id...	Bingo Twin	1 → 1
R9	apply(t(\$1), 2, rev)	Rot90	1 → 1
Y!	matrix(\$3, nrow=\$2, ncol=\$1)	Create Matrix	3 → 1
Y*	\$2 %*% \$1	Matrix Mult	2 → 1
YM	matrix(\$3, nrow=\$2, ncol=\$1, ...	Matrix ByRow	3 → 1
v2	solve(\$2, \$1)	Solve 2	2 → 1
vC	col(\$1)	Col Indices	1 → 1
vE	rowSums(\$1)	Row Sums	1 → 1
vL	solve(\$1)	Solve	1 → 1
vM	colMeans(\$1)	Col Means	1 → 1
vN	rowMeans(\$1)	Row Means	1 → 1
vQ	qr(\$1)	QR Decomp	1 → 1
vR	row(\$1)	Row Indices	1 → 1
vS	colSums(\$1)	Col Sums	1 → 1
vV	svd(\$1)	SVD	1 → 1
y!	diag(\$1)	Diag	1 → 1
yD	diag(\$1)	Identity Matrix	1 → 1
yc	eigen(\$1)\$vectors	EigenVectors	1 → 1
yd	det(\$1)	Determinant	1 → 1
yf	as.matrix(\$1)	Full	1 → 1
yk	kronecker(\$2, \$1)	Kronecker	2 → 1
yl	{x<-\$1; x[upper.tri(x)]<-0; x}	TriLower	1 → 1
ym	matrix(\$3, nrow=\$2, ncol=\$1)	Create Matrix	3 → 1
yt	sum(diag(\$1))	Trace	1 → 1
yu	{x<-\$1; x[lower.tri(x)]<-0; x}	TriUpper	1 → 1

Symbol	R Code	Description	Stack Effect
yv	eigen(\$1)\$values	EigenValues	1 → 1

8.7 Statistics

Symbol	R Code	Description	Stack Effect
B5	fivenum(\$1)	FiveNum	1 → 1
BC	cov(\$2, \$1)	Covariance	2 → 1
BS	summary(\$1)	Summary	1 → 1
Ba	mad(\$1)	MAD	1 → 1
Bc	cor(\$2, \$1)	Correlation	2 → 1
Bi	IQR(\$1)	IQR	1 → 1
Bn	min(\$1)	Min	1 → 1
Br	range(\$1)	Range	1 → 1
Bs	sd(\$1)	Standard Deviation	1 → 1
Bx	max(\$1)	Max	1 → 1
Bz	scale(\$1)	Scale	1 → 1
Ku	sum((\$1-mean(\$1))^4)/((length...	Kurtosis	1 → 1
P	prod(\$1)	Product	1 → 1
Q	quantile(\$1)	Quantile	1 → 1
Sd	ratl_cum_sd(\$1)	CumSD	1 → 1
Sk	sum((\$1-mean(\$1))^3)/((length...	Skewness	1 → 1
Sm	mean(\$1, na.rm=TRUE)	MeanNA	1 → 1
Sn	sum(\$1, na.rm=TRUE)	SumNA	1 → 1
Sw	weighted.mean(\$2, \$1)	WeightedMean	2 → 1
V	var(\$1)	Variance	1 → 1
Xo	ratl_mode(\$1)	Mode	1 → 1
Xr	ratl_rms(\$1)	RMS	1 → 1
Xs	sd(\$1)	Std Dev	1 → 1
cr	cor(\$2, \$1)	Correlation	2 → 1
cv	cov(\$2, \$1)	Covariance	2 → 1
dT	table(\$1)	Table	1 → 1
dX	xtabs(\$2, \$1)	XTabs	2 → 1
h	median(\$1)	Median	1 → 1
lA	any(\$1)	Any	1 → 1
lZ	sum(\$1 != 0)	NNZ	1 → 1
m	mean(\$1)	Mean	1 → 1
mQ	quantile(\$1, \$2)	Quantile	2 → 1
mn	min(\$2, \$1)	Min2	2 → 1
mx	max(\$2, \$1)	Max2	2 → 1
r	runif(\$1)	Random (n)	1 → 1
rm	ratl_rms(\$1)	RMS	1 → 1
s	sum(\$1)	Sum	1 → 1
sd	sd(\$1)	Standard Deviation	1 → 1
sm	summary(\$1)	Summary	1 → 1
v	var(\$1)	Variance	1 → 1
vA	all(\$1)	All	1 → 1
vd	density(\$1)	Density	1 → 1

8.8 Statistical Modeling

Symbol	R Code	Description	Stack Effect
K!	<code>coef(\$1)</code>	Coef	1 → 1
KA	<code>aov(\$1)</code>	AOV	1 → 1
KB	<code>BIC(\$1)</code>	BIC	1 → 1
KE	<code>residuals(\$1)</code>	Residuals	1 → 1
KF	<code>fitted(\$1)</code>	Fitted	1 → 1
KL	<code>logLik(\$1)</code>	LogLik	1 → 1
KV	<code>vcov(\$1)</code>	VCov	1 → 1
av	<code>anova(\$1)</code>	Anova	1 → 1
dA	<code>aggregate(\$3, \$2, \$1)</code>	Aggregate	3 → 1
dE	<code>cut(\$2, \$1)</code>	Cut	2 → 1
dY	<code>by(\$3, \$2, \$1)</code>	By	3 → 1
ka	<code>anova(\$1)</code>	Anova	1 → 1
ke	<code>predict(\$2, \$1)</code>	Predict	2 → 1
kg	<code>glm(\$1)</code>	GLM	1 → 1
ki	<code>AIC(\$1)</code>	AIC	1 → 1
kl	<code>lm(\$1)</code>	Linear Model	1 → 1
kn	<code>nls(\$1)</code>	NLS	1 → 1
ko	<code>loess(\$1)</code>	Loess	1 → 1
lm	<code>lm(\$1)</code>	Linear Model	1 → 1
mC	<code>confint(\$1)</code>	Conf Int	1 → 1
mL	<code>lm(\$2 ~ \$1)</code>	LM Simple	2 → 1
mM	<code>model.matrix(\$1)</code>	Model Matrix	1 → 1
mO	<code>offset(\$1)</code>	Offset	1 → 1
mP	<code>predict(\$1)</code>	Predict	1 → 1
mR	<code>model.frame(\$1)</code>	Model Frame	1 → 1
mT	<code>terms(\$1)</code>	Terms	1 → 1
mU	<code>update(\$2, \$1)</code>	Update	2 → 1
mW	<code>formula(\$1)</code>	Formula	1 → 1
vp	<code>prcomp(\$1)</code>	PCA	1 → 1

8.9 Distributions & Tests

Symbol	R Code	Description	Stack Effect
DN	<code>dnorm(\$3, \$2, \$1)</code>	Density Normal	3 → 1
KC	<code>cor.test(\$2, \$1)</code>	Cor Test	2 → 1
KK	<code>kruskal.test(\$1)</code>	Kruskal-Wallis	1 → 1
N	<code>rnorm(\$1)</code>	Random Normal	1 → 1
PN	<code>pnorm(\$3, \$2, \$1)</code>	Prob Normal	3 → 1
QN	<code>qnorm(\$3, \$2, \$1)</code>	Quantile Normal	3 → 1
RB	<code>rbeta(\$3, \$2, \$1)</code>	Random Beta	3 → 1
RE	<code>rexp(\$2, \$1)</code>	Random Exponential	2 → 1
RN	<code>rnorm(\$3, \$2, \$1)</code>	Random Normal	3 → 1
RU	<code>runif(\$3, \$2, \$1)</code>	Random Uniform	3 → 1
dB	<code>dbinom(\$3, \$2, \$1)</code>	Density Binomial	3 → 1
dC	<code>dchisq(\$2, \$1)</code>	Density Chi-Square	2 → 1
dG	<code>dgeom(\$2, \$1)</code>	Density Geometric	2 → 1
dL	<code>dlogis(\$3, \$2, \$1)</code>	Density Logistic	3 → 1
dN	<code>dnbinom(\$3, \$2, \$1)</code>	Density Neg-Binomial	3 → 1

Symbol	R Code	Description	Stack Effect
dP	dpois(\$2, \$1)	Density Poisson	2 → 1
db	dbeta(\$3, \$2, \$1)	Density Beta	3 → 1
dc	dcauchy(\$3, \$2, \$1)	Density Cauchy	3 → 1
de	dexp(\$2, \$1)	Density Exponential	2 → 1
df	df(\$3, \$2, \$1)	Density F	3 → 1
dg	dgamma(\$3, \$2, \$1)	Density Gamma	3 → 1
dh	dhyper(\$4, \$3, \$2, \$1)	Density Hypergeometric	4 → 1
dl	dlnorm(\$3, \$2, \$1)	Density Log-Normal	3 → 1
dn	dnorm(\$1)	Density Normal	1 → 1
dt	dt(\$2, \$1)	Density Student-t	2 → 1
du	dunif(\$3, \$2, \$1)	Density Uniform	3 → 1
kb	bartlett.test(\$1)	Bartlett Test	1 → 1
kc	chisq.test(\$1)	Chi-Square Test	1 → 1
kf	fisher.test(\$1)	Fisher Test	1 → 1
kk	ks.test(\$2, \$1)	KS Test	2 → 1
kp	prop.test(\$2, \$1)	Prop Test	2 → 1
ks	shapiro.test(\$1)	Shapiro-Wilk	1 → 1
kt	t.test(\$2, \$1)	T-Test	2 → 1
kv	var.test(\$2, \$1)	F-Test Var	2 → 1
kW	wilcox.test(\$2, \$1)	Wilcoxon Test	2 → 1
pB	pbinom(\$3, \$2, \$1)	Prob Binomial	3 → 1
pC	pchisq(\$2, \$1)	Prob Chi-Square	2 → 1
pG	pgeom(\$2, \$1)	Prob Geometric	2 → 1
pL	plogis(\$3, \$2, \$1)	Prob Logistic	3 → 1
pN	pnbinom(\$3, \$2, \$1)	Prob Neg-Binomial	3 → 1
pP	ppois(\$2, \$1)	Prob Poisson	2 → 1
pb	pbeta(\$3, \$2, \$1)	Prob Beta	3 → 1
pc	pcauchy(\$3, \$2, \$1)	Prob Cauchy	3 → 1
pe	pexp(\$2, \$1)	Prob Exponential	2 → 1
pf	pf(\$3, \$2, \$1)	Prob F	3 → 1
pg	pgamma(\$3, \$2, \$1)	Prob Gamma	3 → 1
ph	phyper(\$4, \$3, \$2, \$1)	Prob Hypergeometric	4 → 1
pl	plnorm(\$3, \$2, \$1)	Prob Log-Normal	3 → 1
pn	pnorm(\$1)	Prob Normal	1 → 1
pt	pt(\$2, \$1)	Prob Student-t	2 → 1
pu	punif(\$3, \$2, \$1)	Prob Uniform	3 → 1
pw	pweibull(\$3, \$2, \$1)	Prob Weibull	3 → 1
qB	qbinom(\$3, \$2, \$1)	Quantile Binomial	3 → 1
qC	qchisq(\$2, \$1)	Quantile Chi-Square	2 → 1
qG	qgeom(\$2, \$1)	Quantile Geometric	2 → 1
qL	qlogis(\$3, \$2, \$1)	Quantile Logistic	3 → 1
qN	qnbinom(\$3, \$2, \$1)	Quantile Neg-Binomial	3 → 1
qP	qpois(\$2, \$1)	Quantile Poisson	2 → 1
qb	qbeta(\$3, \$2, \$1)	Quantile Beta	3 → 1
qc	qcauchy(\$3, \$2, \$1)	Quantile Cauchy	3 → 1
qe	qexp(\$2, \$1)	Quantile Exponential	2 → 1
qf	qf(\$3, \$2, \$1)	Quantile F	3 → 1
qg	qgamma(\$3, \$2, \$1)	Quantile Gamma	3 → 1
qh	qhyper(\$4, \$3, \$2, \$1)	Quantile Hypergeometric	4 → 1
ql	qlnorm(\$3, \$2, \$1)	Quantile Log-Normal	3 → 1
qn	qnorm(\$1)	Quantile Normal	1 → 1
qt	qt(\$2, \$1)	Quantile Student-t	2 → 1
qu	qunif(\$3, \$2, \$1)	Quantile Uniform	3 → 1
qw	qweibull(\$3, \$2, \$1)	Quantile Weibull	3 → 1
rB	rbinom(\$3, \$2, \$1)	Random Binomial	3 → 1
rC	rchisq(\$2, \$1)	Random Chi-Square	2 → 1
rG	rgeom(\$2, \$1)	Random Geometric	2 → 1

Symbol	R Code	Description	Stack Effect
rL	rlogis(\$3, \$2, \$1)	Random Logistic	3 → 1
rN	rnbinom(\$3, \$2, \$1)	Random Neg-Binomial	3 → 1
rP	rpois(\$2, \$1)	Random Poisson	2 → 1
rb	rbinom(\$1, size=1, prob=0.5)	Random Binomial	1 → 1
rc	rcauchy(\$3, \$2, \$1)	Random Cauchy	3 → 1
re	rexp(\$1)	Random Exp	1 → 1
rf	rf(\$3, \$2, \$1)	Random F	3 → 1
rg	rgamma(\$3, \$2, \$1)	Random Gamma	3 → 1
rh	rhyper(\$4, \$3, \$2, \$1)	Random Hypergeometric	4 → 1
rl	rlnorm(\$3, \$2, \$1)	Random Log-Normal	3 → 1
rp	rpois(\$1, lambda=1)	Random Poisson	1 → 1
rt	rt(\$2, \$1)	Random Student-t	2 → 1
rw	rweibull(\$3, \$2, \$1)	Random Weibull	3 → 1
tt	t.test(\$2, \$1)	T-Test	2 → 1

8.10 Math Functions

Symbol	R Code	Description	Stack Effect
SP	sinpi(\$1)	Sin(pi*x)	1 → 1
a2	atan2(\$2, \$1)	ArcTan2	2 → 1
aC	acos(\$1)	ArcCos	1 → 1
aS	asin(\$1)	ArcSin	1 → 1
aT	atan(\$1)	ArcTan	1 → 1
ab	abs(\$1)	Abs	1 → 1
ac	acosh(\$1)	ArcCosh	1 → 1
as	asinh(\$1)	ArcSinh	1 → 1
at	atanh(\$1)	ArcTanh	1 → 1
ch	cosh(\$1)	Cosh	1 → 1
cl	ceiling(\$1)	Ceiling	1 → 1
cp	cospi(\$1)	Cos(pi*x)	1 → 1
e-	expm1(\$1)	Exp(x)-1	1 → 1
fl	floor(\$1)	Floor	1 → 1
l+	log1p(\$1)	Log(1+x)	1 → 1
l1	log10(\$1)	Log 10	1 → 1
l2	log2(\$1)	Log 2	1 → 1
lg	log(\$1)	Log Natural	1 → 1
mc	cos(\$1)	Cos	1 → 1
ms	sin(\$1)	Sin	1 → 1
mt	tan(\$1)	Tan	1 → 1
ro	round(\$1)	Round	1 → 1
sg	sign(\$1)	Sign	1 → 1
sh	sinh(\$1)	Sinh	1 → 1
sq	sqrt(\$1)	Sqrt	1 → 1
th	tanh(\$1)	Tanh	1 → 1
tp	tanpi(\$1)	Tan(pi*x)	1 → 1
tr	trunc(\$1)	Trunc	1 → 1

8.11 Combinatorics & Special

Symbol	R Code	Description	Stack Effect
LG	<code>lgamma(\$1)</code>	Log Gamma	1 → 1
MC	<code>choose(\$2, \$1)</code>	Choose	2 → 1
XP	<code>ratl_prime_factors(\$1)</code>	Prime Factors	1 → 1
Xn	<code>choose(\$2, \$1)</code>	Choose (nCr)	2 → 1
Xq	<code>ratl_is_prime(\$1)</code>	Is Prime	1 → 1
di	<code>digamma(\$1)</code>	Digamma	1 → 1
fa	<code>ratl_factors(\$1)</code>	Factors	1 → 1
fp	<code>factorial(\$1)</code>	Factorial	1 → 1
g	<code>ratl_gcd(\$2, \$1)</code>	GCD	2 → 1
lb	<code>lbeta(\$2, \$1)</code>	Log Beta	2 → 1
lc	<code>ratl_lcm(\$2, \$1)</code>	LCM	2 → 1
mB	<code>beta(\$2, \$1)</code>	Beta	2 → 1
mG	<code>gamma(\$1)</code>	Gamma	1 → 1
mf	<code>ratl_prime_factors(\$1)</code>	Prime Factors	1 → 1
ml	<code>ratl_lcm(\$2, \$1)</code>	LCM	2 → 1
mp	<code>ratl_is_prime(\$1)</code>	Is Prime	1 → 1
ps	<code>psigamma(\$2, \$1)</code>	Psigamma	2 → 1
tg	<code>trigamma(\$1)</code>	Trigamma	1 → 1

8.12 Complex Numbers

Symbol	R Code	Description	Stack Effect
cA	<code>Arg(\$1)</code>	Arg	1 → 1
cI	<code>Im(\$1)</code>	Imag Part	1 → 1
cJ	<code>Conj(\$1)</code>	Conjugate	1 → 1
cM	<code>Mod(\$1)</code>	Modulus	1 → 1
cR	<code>Re(\$1)</code>	Real Part	1 → 1

8.13 String Operations

Symbol	R Code	Description	Stack Effect
C	<code>paste0(\$2, \$1)</code>	Concat	2 → 1
J	<code>paste(\$2, collapse=\$1)</code>	Join List	2 → 1
S	<code>strsplit(\$2, \$1)[[1]]</code>	Split String	2 → 1
S!	<code>chartr(\$2, \$1, \$3)</code>	Translate	3 → 1
SG	<code>gsub(\$2, \$1, \$3)</code>	GSub	3 → 1
Xc	<code>chartr(\$1, \$2, \$3)</code>	Char Translate	3 → 1
Xt	<code>tolower(\$1)</code>	ToLower	1 → 1
j	<code>paste(\$2, \$1)</code>	Join	2 → 1
rv	<code>paste(rev(strsplit(as.character...</code>	Reverse String	1 → 1
sB	<code>grepl(\$2, \$1)</code>	Grepl	2 → 1
sG	<code>grep(\$2, \$1)</code>	Grep	2 → 1

Symbol	R Code	Description	Stack Effect
sL	<code>tolower(\$1)</code>	ToLower	1 → 1
sR	<code>regexpr(\$2, \$1)</code>	Regexpr	2 → 1
sU	<code>toupper(\$1)</code>	ToUpper	1 → 1
sX	<code>gregexpr(\$2, \$1)</code>	Gregexpr	2 → 1
sf	<code>sprintf(\$2, \$1)</code>	Sprintf	2 → 1
sn	<code>nchar(\$1)</code>	NChar	1 → 1
sr	<code>sub(\$2, \$1, \$3)</code>	Sub	3 → 1
ss	<code>substr(\$3, \$2, \$1)</code>	Substr	3 → 1
st	<code>trimws(\$1)</code>	TrimWS	1 → 1

8.14 Set Operations

Symbol	R Code	Description	Stack Effect
sD	<code>setdiff(\$2, \$1)</code>	SetDiff	2 → 1
sI	<code>intersect(\$2, \$1)</code>	Intersect	2 → 1
sN	<code>union(\$2, \$1)</code>	Union	2 → 1

8.15 Bitwise Operations

Symbol	R Code	Description	Stack Effect
bA	<code>bitwAnd(\$2, \$1)</code>	BitAnd	2 → 1
bL	<code>bitwShiftL(\$2, \$1)</code>	BitShiftL	2 → 1
bN	<code>bitwNot(\$1)</code>	BitNot	1 → 1
bO	<code>bitwOr(\$2, \$1)</code>	BitOr	2 → 1
bR	<code>bitwShiftR(\$2, \$1)</code>	BitShiftR	2 → 1
bX	<code>bitwXor(\$2, \$1)</code>	BitXor	2 → 1

8.16 Type & Introspection

Symbol	R Code	Description	Stack Effect
A	<code>utf8ToInt(\$1)</code>	To ASCII	1 → 1
AS	<code>as(\$2, \$1)</code>	As Type	2 → 1
Xb	<code>ratl_bin(\$1)</code>	To Binary	1 → 1
Xd	<code>as.Date(\$1)</code>	To Date	1 → 1
a	<code>as.character(\$1)</code>	To String	1 → 1
c	<code>intToUtf8(\$1)</code>	To Char	1 → 1
ev	<code>NULL</code>	Eval	1 → 0
is	<code>is(\$2, \$1)</code>	Is Type	2 → 1
mF	<code>factor(\$1)</code>	Factor	1 → 1

Symbol	R Code	Description	Stack Effect
n	as.numeric(\$1)	To Number	1 → 1
oA	attr(\$2, \$1)	Attr	2 → 1
oC	colnames(\$1)	Colnames	1 → 1
oa	attributes(\$1)	Attributes	1 → 1
oc	class(\$1)	Class	1 → 1
od	dim(\$1)	Dim	1 → 1
on	names(\$1)	Names	1 → 1
or	rownames(\$1)	Rownames	1 → 1
ot	typeof(\$1)	typeof	1 → 1
ou	unlist(\$1)	Unlist	1 → 1
zB	NULL	Sort By	2 → 1
zC	NULL	Scan	2 → 1
zD	NULL	Dispatch List	0 → 1
zG	NULL	Group By	2 → 1
zS	NULL	Send	1 → 1
zT	NULL	Take While	2 → 1
zW	NULL	Drop While	2 → 1
zX	NULL	Shell Exec	1 → 1
zZ	NULL	Zip With	3 → 1
zd	as.Date(\$1)	To Date	1 → 1

8.17 File I/O

Symbol	R Code	Description	Stack Effect
F	readLines(\$1)	Read File	1 → 1
FL	list.files(\$1)	List Files	1 → 1
FW	{writeLines(as.character(\$2), ...	Write File	2 → 0
W	writeLines(as.character(\$2), ...	Write File	2 → 0
fA	dirname(\$1)	Dirname	1 → 1
fC	write.csv(\$2, \$1)	Write CSV	2 → 0
fD	list.dirs(\$1)	List Dirs	1 → 1
fN	file.create(\$1)	File Create	1 → 1
fP	normalizePath(\$1)	Abs Path	1 → 1
fT	tempfile()	Temp File	0 → 1
fX	dir.exists(\$1)	Dir Exists	1 → 1
fb	basename(\$1)	Basename	1 → 1
fc	read.csv(\$1)	Read CSV	1 → 1
fd	dir.create(\$1)	Dir Create	1 → 1
fe	file.exists(\$1)	File Exists	1 → 1
fi	file.info(\$1)	File Info	1 → 1
fm	file.remove(\$1)	File Remove	1 → 1
fr	read.table(\$1)	Read Table	1 → 1
ft	tempdir()	Temp Dir	0 → 1
fw	write.table(\$2, \$1)	Write Table	2 → 0

8.18 System

Symbol	R Code	Description	Stack Effect
T	<code>as.numeric(Sys.time())</code>	Time Epoch	0 → 1
ZT	<code>Sys.timezone()</code>	Timezone	0 → 1
ge	<code>Sys.getenv(\$1)</code>	Getenv	1 → 1
pi	<code>Sys.getpid()</code>	GetPID	0 → 1
sE	<code>date()</code>	Date	0 → 1
sF	<code>options(prompt=\$1)</code>	SetPrompt	1 → 0
sH	<code>getwd()</code>	GetWD	0 → 1
sJ	<code>setwd(\$1)</code>	SetWD	1 → 0
sK	<code>proc.time() - \$1</code>	Toc	1 → 1
sP	<code>Sys.sleep(\$1)</code>	Sleep	1 → 0
sT	<code>proc.time()</code>	Tic	0 → 1
sV	<code>R.version.string</code>	RVersion	0 → 1
sY	<code>options(scipen=\$1)</code>	SetScipen	1 → 0
se	<code>Sys.setenv(...)</code>	Setenv	1 → 0
um	<code>Sys.umask(\$1)</code>	Umask	1 → 1
zg	<code>gc()</code>	GC	0 → 1
zl	<code>Sys.localeconv()</code>	Locale	0 → 1
zo	<code>options()</code>	Options	0 → 1
zt	<code>Sys.time()</code>	SysTime	0 → 1
zv	<code>version</code>	Version	0 → 1